NAME:

Project 3: Wordle (5 pts)

In this project, you'll implement variants of a text-based Wordle game. If you're not familiar with the game, check out the Wikipedia article about it: (https://en.wikipedia.org/wiki/Wordle).

In the past few years Wordle has become a popular game (https://www.nytimes.com/games/wordle/index.html with increasingly complex strategies described by emerging jargon (https://www.nytimes.com/2023/08/01/crosswords/how-to-talk-about-wordle.html). You will create a simplified Wordle game using only Python lists.

Learning Objectives:

1. Demonstrate that you can use the programming concepts that we've covered so far. In particular: lists including nested lists.
2. Practice writing interactive programs.
3. Explore ways computer use their interactions with humans to begin to "learn".

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Base Wordle

Create a simplified Wordle game using only Python lists.

- Create a python file named `base_wordle.py`
- Create a list of 20 five-letter words. These words should be stored as a list of a lists named `word_list`
  - Each word is a list element, such that each character is a list element

```
word_list = [
  ['G', 'R', 'E', 'A', 'T'],
  ['C', 'R', 'A', 'N', 'E'],
  ...
]
print(word_list[0][0]) # should print "G"
```

- For now, hardcode the correct answer
- Allow the player up to 6 guesses.
- For each guess (a list of characters), provide feedback for each letter:
- Print the guess alongside its feedback after every attempt.
- End the game if the player guesses the word correctly or after 6 tries.
  - If the player guesses, the player wins!
  - If the player does not guess in 6 tries, the computer wins! reveal the secret word to the player

Example output, if the actual word is `GREAT`:

```
Your Guess: CRANE
Feedback: 'R' is at the correct location, 'A', 'E' are in the word, but at the
wrong location. Try again!

Enter your guess:
```

If the user guesses correctly, display: `"You got it!"`

2. Random Wordle

- Start a new python file named `random_wordle.py` and copy in your previous code
- Update your code so that the word is chosen randomly from the list

3. Learning Wordle

- Start a new python file names `learning_wordle.py` and copy your previous code
- Now make it so that every time a user enters a word, the word gets added to your `word_list`

3.2 Write a reflection:

- The "learning" version of Wordle will create copies of some words in your `word_list`, i.e. some words will be duplicated. By doing this you are simulating a real-world Machine-Learning like scenario:
- You want to offer a real chance for your human players to win, so the computer is trying to "learn" what are some common words people start with
- Because you maybe using the random module to pick a random word from word_list to be the correct word, the probability that the word that occurs multiple times throughout the list is picked as the correct word increases.
- This is an over-simplified version of how ChatGPT works!! Write a short reflection on:
  - Based on your experience writing this code: What is your guess? What does ChatGPT do?
  - Does this update your answer to "Can computers think?"
  - Save your reflection as `wordle_reflection.pdf`.

3. What to submit

What to submit You will need to submit the following files:

- `base_wordle.py`
- `random_wordle.py`
- `learning_wordle.py`
- `wordle_reflection.pdf`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:

a. A header comment, which includes your name and a brief description of the program.
b. Comments within the body of the code to further clarify how the program works.

2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code? Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?

a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting. Once you have checked these things, submit your files to *cat3_Project 3: Wordle* on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

7. Demonstration Instructions

- TBD